ArgiSense Methane + Pressure Firmware Overview

argisense_mp_u575rg board, Zephyr 4.4.0, RS485 Modbus RTU, RS485 DFU, USB-C update, HTU21D, MS5803, GP8302

Updated: 2026-06-27

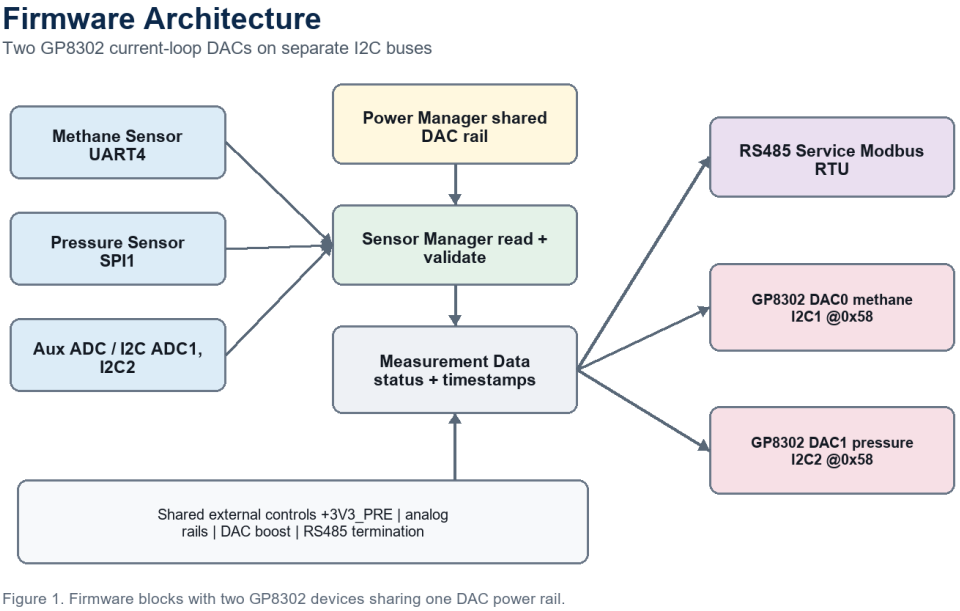

Figure 1. Firmware architecture and module boundaries

This document describes the current firmware architecture and remaining integration notes for the ArgiSense methane + pressure product profile on the `argisense_mp_u575rg` board, based on STM32U575RGT6 and Zephyr 4.4.0.

The firmware reads methane gas from a Dynament Platinum Hydrocarbon infrared sensor, pressure from an MS580305BA01-00 sensor, and humidity/ambient temperature from an HTU21D sensor. It publishes measurements over RS485 and drives two 4-20 mA analog outputs through two GP8302 current DAC devices. The design is power-first, while keeping the Dynament methane sensor powered continuously by default.

Product Board Split

This document applies to the methane + pressure product board:

```
argisense_mp_u575rg
```

The pH product uses a separate board target because its PCB and pinout are different:

```
argisense_ph_u575rg
```

Recommended firmware split:

Product	Board target	Firmware profile	Notes
Methane + pressure	argisense_mp_u575rg	Current document	Two sensors, RS485, DAC0 methane, DAC1 pressure
pH	argisense_ph_u575rg	Future pH profile	pH front-end, pH compensation, pH-specific DAC mapping

Shared services such as MCUboot, RS485 framing, calibration storage, power sequencing, and 4-20 mA scaling should stay common. Sensor measurement logic and DAC source mapping should be product-specific.

Firmware Goals

- Periodically measure methane gas concentration.
- Periodically measure pressure.
- Periodically measure relative humidity and ambient temperature with HTU21D.
- Export live values and status over RS485.
- Drive two 4-20 mA outputs: GP8302 DAC0 for methane and GP8302 DAC1 for pressure.
- Keep high-current rails off whenever the board is idle.
- Preserve a clean path for calibration, diagnostics, and future OTA/storage.

Methane + Pressure Board Interfaces

The `argisense_mp_u575rg` board DTS currently exposes these hardware interfaces:

Function	Zephyr device	MCU pins	Schematic role
Debug console	usart1	PA9, PA10	Console/debug UART
RS485	usart2	PA2, PA3, PA1 DE	Half-duplex RS485 port
Methane sensor	uart4	PC10, PC11	Dynament Platinum methane UART, 38400 baud
DAC0 and EEPROM bus	i2c1	PB8, PB9	Methane GP8302 at 0x58 and EEPROM
External EEPROM	i2c1	PB8, PB9	AT24C512C at 0x50, 64 KiB, Zephyr atmel,at24 driver
DAC1/helper bus	i2c2	PB10, PB14	Pressure GP8302 at 0x58, analog monitor, and HTU21D
Humidity sensor	i2c2	PB10, PB14	HTU21D at 0x40, argisense,htu21d driver
Pressure sensor	spi1	PA5, PA6, PA7, PA4 CS	MS580305BA01-00 / MS5803-05BA pressure sensor SPI
Thermistor/ADC	adc1	PC0 / ADC1_IN1	Analog temperature or board sensing

The board-specific GPIOs are exposed through `/zephyr,user`:

Property	Purpose	Default firmware state
dac-channel-count	Declares the firmware DAC output count	2
dac0-output-source	Documents GP8302 DAC0 source on i2c1	methane
dac1-output-source	Documents GP8302 DAC1 source on i2c2	pressure
pressure-part-number	Documents the selected pressure sensor	MS580305BA01-00
pressure-interface	Documents the selected pressure sensor bus	spi
pressure-ps-active-state	Documents protocol-select polarity for the selected pressure sensor	low
humidity-part-number	Documents the selected humidity sensor	HTU21D
humidity-interface	Documents the selected	i2c2

	humidity sensor bus	
pre-power-gpios	Enables +3V3_PRE sensor rail	Off
analog-power-gpios	Enables analog positive/negative rail section	Off
dac-power-gpios	Enables the shared DAC/4-20 mA boost supply for both GP8302 devices	Off
dac-alarm-gpios	Reads DAC alarm/status pin if populated as a shared alarm	Input
rs485-termination-gpios	Enables optional 120 ohm termination	Off
pressure-ps-gpios	MS5803 protocol select, low = SPI and high = I2C	Low during SPI measurements
pressure-cs-gpios	Pressure sensor SPI chip select	Inactive

Runtime Model

The firmware runs as one primary measurement loop plus small service modules. Hardware protocol code lives in out-of-tree Zephyr drivers, while product policy, persistent settings, Modbus registers, and shell commands stay in `app/src`.

Current split:

Component	Location	Responsibility
Main loop	app/src/main.c	Coordinates power, sensor reads, DAC updates, register snapshots, and MCUboot confirmation
Power manager	app/src/argisense_power.c	Controls sensor rails, DAC power, RS485 termination, and pressure protocol-select GPIOs
Settings	app/src/argisense_settings.c	Loads and saves runtime configuration through Zephyr Settings/NVS
Register model	app/src/argisense_registers.c	Owns the Modbus holding-register map and coherent measurement snapshots
RS485 service	app/src/argisense_rs485.c	Starts the Zephyr Modbus RTU server and forwards reads/writes to the register model
USB-C service	app/src/argisense_usb.c, snippets/argisense-usb-update	Owns the composite CDC ACM USB device profile for console, shell, and MCUmgr firmware update
Shell diagnostics	app/src/argisense_shell.c	Provides driver, sensor, settings, and register inspection commands
Current-loop output	app/src/current_loop_output.c	Converts methane and pressure process values to GP8302 current commands
Methane driver	drivers/sensor/dynament_platinum	Dynament UART live-data-simple request/parser exposed through Zephyr Sensor API
Pressure driver	drivers/sensor/ms5803_05ba	MS5803 reset, PROM CRC, ADC conversion, and pressure/temperature compensation
Humidity driver	drivers/sensor/htu21d	HTU21D reset, humidity/temperature conversion, CRC check, and Sensor API channels
DAC driver	drivers/dac	GP8302 Zephyr DAC API implementation
External EEPROM	Zephyr drivers/eeprom/eeprom_at2x.c	AT24C512C access through the standard EEPROM API

Important files:

```
app/src/main.c
app/src/argisense_power.c
app/src/argisense_power.h
app/src/argisense_settings.c
app/src/argisense_settings.h
app/src/argisense_registers.c
```

```
app/src/argisense_registers.h
app/src/argisense_rs485.c
app/src/argisense_rs485.h
app/src/argisense_usb.c
app/src/argisense_shell.c
app/src/current_loop_output.c
app/src/current_loop_output.h
drivers/sensor/dynament_platinum/
drivers/sensor/ms5803_05ba/
drivers/sensor/htu21d/
drivers/dac/
dts/bindings/sensor/argisense,htu21d.yaml
snippets/argisense-usb-update/
tools/usb_mcumgr/
tools/rs485_dfu/
```

Measurement Cycle

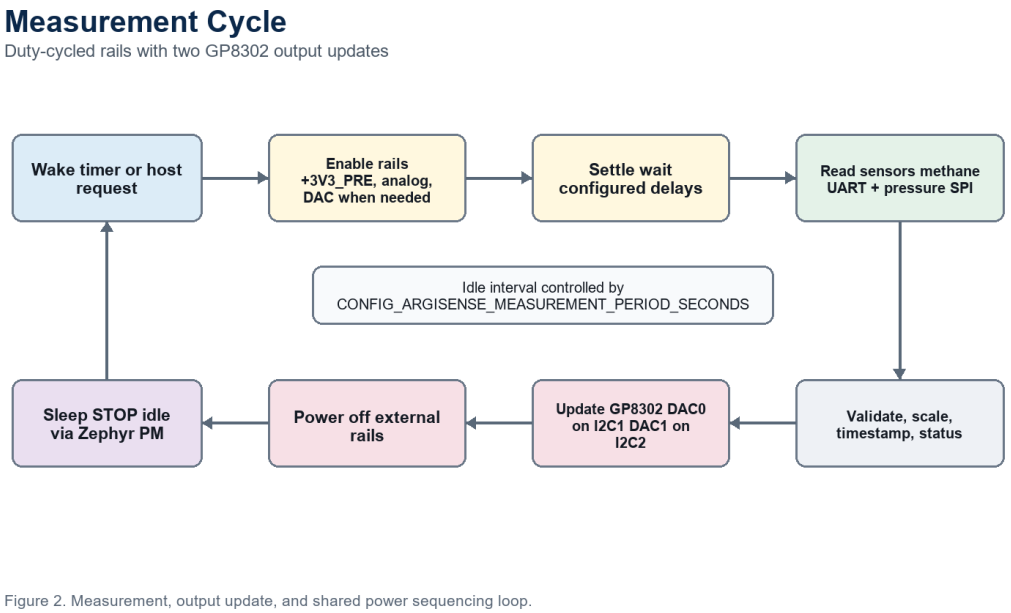

Figure 2. Measurement, output update, and shared power sequencing loop.

Figure 2. Measurement cycle and power sequencing

The measurement sequence is:

1. Wake from Zephyr idle.
2. Keep RS485 termination disabled unless configured by the host.
3. Enable `+3V3_PRE`.
4. Wait `CONFIG_ARGISENSE_PRE_RAIL_SETTLE_MS`.
5. Configure pressure `PS` mode.
6. Enable analog rails.
7. Wait `CONFIG_ARGISENSE_ANALOG_RAIL_SETTLE_MS`.
8. Read pressure over SPI.
9. Read Dynament methane live-data-simple over UART.
10. Read HTU21D humidity and ambient temperature over I2C when the configured humidity read period has elapsed; otherwise reuse the cached humidity sample.

11. Validate and timestamp all readings.
12. Update the internal data model.
13. Update GP8302 DAC0 from methane and GP8302 DAC1 from pressure if enabled.
14. Turn off DAC power when the current loops do not need continuous output.
15. Turn off analog rails.
16. Keep `+3V3_PRE` on by default so the Dynament methane sensor is not

repeatedly power-cycled.

17. Sleep until the next measurement period.

Current Kconfig duty-cycle controls:

```
CONFIG_ARGISENSE_MEASUREMENT_PERIOD_SECONDS
CONFIG_ARGISENSE_PRE_RAIL_ALWAYS_ON
CONFIG_ARGISENSE_PRE_RAIL_SETTLE_MS
CONFIG_ARGISENSE_ANALOG_RAIL_SETTLE_MS
CONFIG_ARGISENSE_MEASUREMENT_WINDOW_MS
CONFIG_ARGISENSE_PRESSURE_PS_ACTIVE
CONFIG_ARGISENSE_DAC_POWER_DURING_MEASUREMENT
CONFIG_ARGISENSE_DAC_MIN_CURRENT_UA
CONFIG_ARGISENSE_DAC_MAX_CURRENT_UA
CONFIG_ARGISENSE_DAC_FAULT_CURRENT_UA
CONFIG_ARGISENSE_METHANE_DAC_RANGE_LOW_PPM
CONFIG_ARGISENSE_METHANE_DAC_RANGE_HIGH_PPM
CONFIG_ARGISENSE_METHANE_SENSOR_WARMUP_SECONDS
CONFIG_ARGISENSE_METHANE_SENSOR_READ_PERIOD_MS
CONFIG_ARGISENSE_HUMIDITY_SENSOR_READ_PERIOD_SECONDS
CONFIG_ARGISENSE_PRESSURE_DAC_RANGE_LOW_PA
CONFIG_ARGISENSE_PRESSURE_DAC_RANGE_HIGH_PA
```

Methane Sensor Handling

The selected methane sensor family is the Dynament Platinum Series Hydrocarbon infrared gas sensor. It is connected to `uart4` at 38400 baud using 8 data bits, 1 stop bit, and no parity. The board DTS declares it as `argisense,dynament-platinum-hydrocarbon`.

Recommended behavior:

- Keep the sensor powered continuously by default. Dynament documents the

Platinum sensor as intended for continuous powered operation, and repeated short-interval power cycling is outside the intended use of the product.

- Allow up to 60 seconds after first power before trusting gas readings. During

warm-up or fault conditions, the digital live data can report `-250`.

- Send the live-data-simple request from AN0007:

```
10 13 06 10 1F 00 58
```

- Parse the compact live-data-simple response:
- `10 1A` means DLE + DAT.
- byte 3 is data length.
- bytes 4-5 are version, little-endian.
- bytes 6-7 are status flags, little-endian.
- bytes 8-11 are the gas reading as little-endian IEEE-754 float.

- `10 1F` means DLE + EOF.
- final two bytes are a high/low additive checksum.
- Treat the gas reading as percent volume and convert it to `ppm_x100` with:

```
ppm_x100 = percent_volume * 1,000,000
```

- Handle Dynament byte-stuffed DLE/control characters in the final UART stream

parser before decoding the compact frame.

- Use a read timeout so the measurement loop cannot hang.
- Track these fields in the data model:
- methane concentration
- methane percent volume
- methane unit
- Dynament status flags
- warm-up state
- last successful read timestamp
- communication error counter

The methane sensor should be treated as invalid during warm-up or after a UART timeout. The RS485 status map and 4-20 mA fault behavior must expose this state. The detailed integration note is in `docs/dynament-platinum-methane.md`.

Humidity Sensor Handling

The selected humidity sensor is HTU21D. It is connected to `i2c2` at address `0x40` and is handled by the out-of-tree `argisense,htu21d` driver under `drivers/sensor/htu21d`. The driver performs soft reset, I2C no-hold humidity/temperature conversions, CRC validation, and Zephyr Sensor API channel conversion.

Implemented behavior:

- Bind the DTS node with `compatible = "argisense,htu21d"` and

`part-number = "HTU21D"`.

- Send soft reset command `0xfe` during driver initialization.
- Use no-hold humidity command `0xf5` and no-hold temperature command `0xf3`.
- Validate the 3-byte HTU21D response with CRC-8 polynomial `0x31`.
- Expose `SENSOR_CHAN_HUMIDITY` and `SENSOR_CHAN_AMBIENT_TEMP` through the

standard Zephyr Sensor API.

- Use `CONFIG_ARGISENSE_HUMIDITY_SENSOR_READ_PERIOD_SECONDS` to avoid reading

humidity more often than needed; the application caches the latest successful sample.

- Store relative humidity as `%RH x100` and ambient temperature as centi-degrees

Celsius in the central data model.

- Expose humidity sensor validity and communication errors in register `2`

status bits.

- Expose HTU21D diagnostics over RS485 registers `80..82`.
- Treat humidity as auxiliary process data; it does not drive either 4-20 mA

output by default.

The detailed integration note is in `docs/htu21d-humidity.md`.

Pressure Sensor Handling

The selected pressure sensor is TE Connectivity `MS580305BA01-00` (`MS5803-05BA01`). It is a 5 bar / 500 kPa absolute digital pressure sensor with a 24-bit ADC and SPI/I2C digital interface. The `argisense_mp_u575rg` board uses SPI on `spi1` with GPIO chip select on `PA4`, so the firmware keeps the sensor `PS` pin low during measurement windows.

Recommended behavior:

- Keep `pressure-ps-gpios` low before enabling SPI transactions.
- Send the MS5803 reset command before the first calibration read.
- Read PROM calibration coefficients and verify the PROM CRC.
- Start D1 pressure conversion and D2 temperature conversion with the selected

oversampling ratio.

- Wait the datasheet conversion time before reading the 24-bit ADC result.
- Apply the MS5803 first-order and second-order compensation equations before

publishing pressure.

- Use pascals in the firmware data model and RS485 register map.
- Track these fields:
- raw pressure sample
- compensated pressure
- pressure unit
- sensor temperature if available
- sensor status
- CRC or data-valid flag if supported by the sensor

Pressure sensor chip select should stay inactive while the sensor rail is off.

Dual GP8302 4-20 mA Outputs

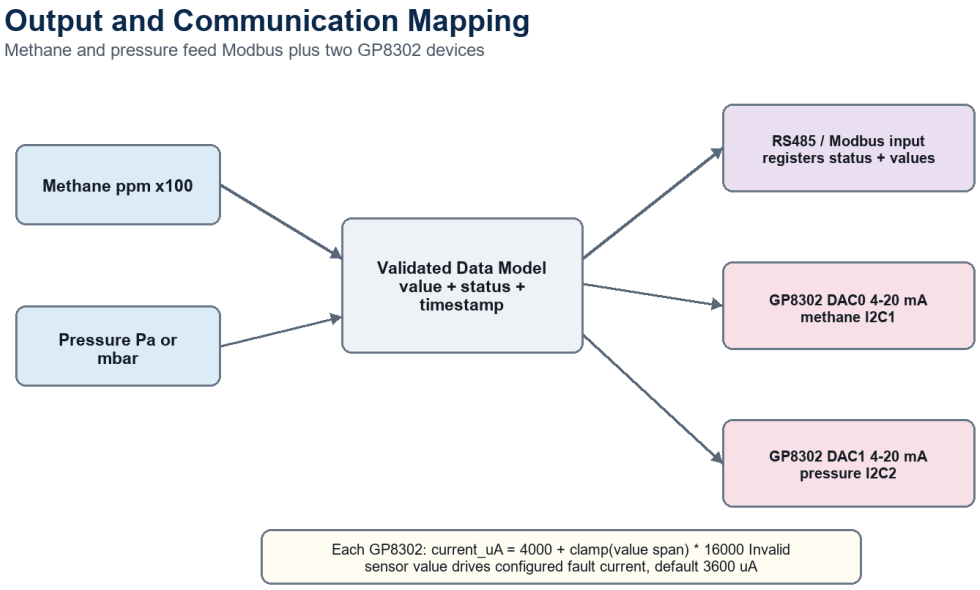

Figure 3. Sensor, communication, and 4-20 mA output mapping

The board uses two single-output GP8302 current DAC devices. They share the same DAC/current-loop power enable but are mapped to different I2C buses so both devices can keep the GP8302 default address.

Default hardware mapping:

Output	GP8302 alias/node	I2C bus	I2C address	Source value	Default low point	Default high point
DAC0	current-loop-dac0 / dac0_gp8302	i2c1	0x58	Methane concentration	CONFIG_ARGISENSE_METHANE_DAC_RANGE_LOW_PPM	CONFIG_ARGISENSE_METHANE_DAC_RANGE_HIGH_PPM
DAC1	current-loop-dac1 / dac1_gp8302	i2c2	0x58	Pressure	CONFIG_ARGISENSE_PRESSURE_DAC_RANGE_LOW_PA	CONFIG_ARGISENSE_PRESSURE_DAC_RANGE_HIGH_PA

The default methane DAC range is 0 ppm to 1000000 ppm, matching a 0-100% volume Dynament methane variant. For a 0-5% volume ordered variant, set the high point to 50000 ppm.

For the selected MS580305BA01-00 pressure sensor, the default pressure DAC range is 0 Pa to 500000 Pa, matching the nominal 5 bar absolute sensor range.

The default `0x58` address follows the common GP8302 module/library default. Using separate buses avoids an address conflict without adding an I2C mux or changing hardware addresses.

Current behavior and policy:

- `CONFIG_ARGISENSE_DAC_POWER_DURING_MEASUREMENT` powers the shared DAC block

while outputs are refreshed.

- `CONFIG_ARGISENSE_DAC_POWER_IDLE` decides whether the shared DAC block remains

powered between measurement cycles.

- Runtime settings provide methane range, pressure range, normal current limits, and fault current.
- Each process value is mapped to its own current command:

```
span = clamp((value - range_low) / (range_high - range_low), 0.0, 1.0)
current_mA = 4.0 + (span * 16.0)
```

- Invalid source values are driven to `dac_fault_current_ua`; the default is 3600 microamps.
- The GP8302 driver converts microamps to a 12-bit raw code and writes channel `0` through the Zephyr DAC API.

Current Kconfig defaults feeding runtime settings:

```
CONFIG_ARGISENSE_DAC_MIN_CURRENT_UA
CONFIG_ARGISENSE_DAC_MAX_CURRENT_UA
CONFIG_ARGISENSE_DAC_FAULT_CURRENT_UA
CONFIG_ARGISENSE_METHANE_DAC_RANGE_LOW_PPM
CONFIG_ARGISENSE_METHANE_DAC_RANGE_HIGH_PPM
CONFIG_ARGISENSE_PRESSURE_DAC_RANGE_LOW_PA
CONFIG_ARGISENSE_PRESSURE_DAC_RANGE_HIGH_PA
CONFIG_ARGISENSE_DAC_POWER_DURING_MEASUREMENT
CONFIG_ARGISENSE_DAC_POWER_IDLE
```

RS485 Communication

The RS485 interface is connected to `usart2` with hardware DE on `PA1`.

Implemented protocol: Modbus RTU slave.

Reasons:

- Common for industrial sensors.
- Easy to test with existing RS485 tools.
- Natural fit for measured values, status words, calibration, and output configuration.

Default electrical and UART setup:

Setting	Default/current value
Interface	RS485 half-duplex
UART	usart2
Baud rate	115200 by default, configurable through settings/register 8
Data bits	8; exposed in settings/registers, currently fixed for Modbus RTU
Parity	0=none by default; configurable as 0=none, 1=odd, 2=even
Stop bits	2 by default for Modbus RTU no-parity framing; configurable as 1 or 2
DE control	STM32 USART hardware DE
Termination	Disabled by default, host configurable

Implemented holding-register map summary:

Register block	Access	Description
0..19	R/RW	Device identity, status, serial settings, live methane/pressure values, DAC

		current commands, sample sequence, and sample uptime
20..29	R	MCUboot image version, boot flags, active slot, reboot-required flag, and last command result
30..36	R/W	DAC current limits, command register, RS485 parity, stop bits, and data bits
40..59	R/W	Methane/pressure DAC ranges, offsets, and DAC trim placeholders
70..82	R	Dynament status/protocol, last sensor errors, MS5803 temperature/raw/CRC diagnostics, and HTU21D humidity/temperature/error diagnostics
1000..1035	R/W	RS485 firmware-update control, status, image metadata, SHA-256, and command registers
1100..	R/W	RS485 firmware-update chunk payload window

The full register table is maintained in `README.md`. All multi-register values use high word first. Masters should read paired 32-bit values in one Modbus request, or compare the sample sequence before and after a larger read block.

RS485 can also be used as the sealed-product service and firmware update path. The PC GUI in `tools/rs485_dfu/argisense_rs485_dfu_gui.py` has tabs for firmware update, live sensor monitoring/graphing, and device configuration. It can auto-detect an ArgiSense node by scanning COM ports, supported baud presets, 8-bit Modbus RTU framing variants, parity, stop bits, and Unit IDs until register `0` returns device ID `0xA651`. The firmware update tab uploads `build/argisense-zephyr-app/zephyr/zephyr.signed.bin` over Modbus holding registers. The application writes the image to MCUboot `image-1`, verifies CRC32 and SHA-256, then can mark the verified image for a test swap and reboot. MCUboot performs the swap on the next boot, and the application confirms the image after bring-up checks pass.

USB-C Firmware Update and Service Console

The board USB-C connector can be built as a service connector during bring-up and factory work. Build with the `argisense-usb-update` snippet or the `usbupdate` helper-script option to enable a composite USB CDC ACM device:

```
CDC ACM 0: Zephyr console, log output, and shell
CDC ACM 1: MCUmgr SMP firmware update transport
```

The MCUmgr port uploads the MCUboot signed binary to the secondary image slot:

```
mcumgr --conntype serial --connstring "dev=COMxx,baud=115200,mtu=128" image list
mcumgr --conntype serial --connstring "dev=COMxx,baud=115200,mtu=128" image upload
build\argisense-zephyr-app\zephyr\zephyr.signed.bin --noerase=false
mcumgr --conntype serial --connstring "dev=COMxx,baud=115200,mtu=128" image test <new-image-hash>
mcumgr --conntype serial --connstring "dev=COMxx,baud=115200,mtu=128" reset
```

The repository also provides a Python/Tkinter GUI for this USB-C update path:

```
py -3.12 -m pip install -r argisense-zephyr-app\tools\usb_mcumgr\requirements.txt
py -3.12 argisense-zephyr-app\tools\usb_mcumgr\argisense_usb_mcumgr_gui.py
```

The GUI selects the MCUmgr CDC ACM port, probes `image list`, parses the selected signed image header, shows image version and size, runs upload/test/reset, and tracks progress from `mcumgr image upload`. Keep `Erase secondary slot before upload` enabled during normal field service. That option passes `--noerase=false` to `mcumgr` and avoids stale data in the secondary slot.

The USB update snippet disables the `mcumgr_img_grp` log module because Zephyr's optional upload `match` check can print a misleading SHA-256 error when the full file hash and MCUboot image hash differ. The image can still be accepted and booted by MCUboot. RS485 DFU keeps explicit CRC32 and SHA-256 verification enabled in the application path.

Current bring-up policy is rollback-friendly: MCUboot downgrade prevention is disabled in sysbuild so field service can install an older known-good firmware when needed. Production builds can re-enable downgrade prevention after the final update policy is fixed.

Data Model

RS485 and DAC output read from the same validated measurement sample. The sample is produced by the measurement loop, then copied into the register snapshot by `argisense_register_update_sample()`.

Current measurement sample fields:

```
struct argisense_measurement_sample {
      int32_t methane_ppm_x100;
      int32_t pressure_pa;
      int32_t methane_last_error;
      int32_t pressure_last_error;
      int32_t humidity_last_error;
      int32_t pressure_temperature_centi_c;
      int32_t humidity_rh_x100;
      int32_t humidity_temperature_centi_c;
      uint32_t pressure_d1_raw;
      uint32_t pressure_d2_raw;
      uint16_t methane_status_flags;
      uint16_t methane_protocol_version;
      uint8_t pressure_prom_crc_read;
      uint8_t pressure_prom_crc_calc;
      bool methane_valid;
      bool pressure_valid;
      bool humidity_valid;
};
```

Holding register `2` currently exposes these status flags:

```
BIT(0) methane_valid
BIT(1) pressure_valid
BIT(2) sample_ready
BIT(3) rs485_termination_enabled
BIT(4) humidity_valid
BIT(5) humidity_error
```

Power Strategy

The board should keep only the MCU, always-on support circuitry, and the Dynament methane sensor rail alive during idle by default.

Idle state:

- `+3V3_PRE` on by default for the Dynament methane sensor.
- Analog rails off.
- Shared DAC boost off unless continuous 4-20 mA output is required.
- RS485 termination off unless the installation requires this node to terminate

the bus.

- Pressure CS inactive.
- Zephyr PM allowed to enter STM32U575 STOP idle states.

Measurement state:

- Ensure the pre-sensor rail is already on or enable it if `CONFIG_ARGISENSE_PRE_RAIL_ALWAYS_ON` is disabled.
- Wait for rails to settle.
- Read sensors.
- Update output and communication data.
- Return rails to off state.

Product decision to confirm: whether either 4-20 mA loop must remain driven while the MCU sleeps. If yes, the shared DAC power strategy must change from duty-cycled to continuous or latched-output mode.

Fault Handling

Minimum recommended fault handling:

- If a sensor read fails, keep the last valid value and mark the corresponding status bit invalid.
- If the same sensor fails for N consecutive cycles, drive that sensor's DAC channel to the configured fault current.
- If either DAC alarm is asserted, expose it over RS485 immediately.
- If rail GPIO setup fails at boot, stop the measurement loop and report a fatal error over console.
- If RS485 receives unsupported commands, respond with a protocol error rather than silently ignoring them.

Calibration and Persistent Settings

The firmware uses Zephyr Settings with the NVS backend in the `storage_partition`. Runtime settings are stored under:

```
argisense/config
```

The stored configuration currently includes:

- Methane offset and span calibration.
- Pressure offset and span calibration.
- Per-channel DAC current trim at 4 mA and 20 mA.
- RS485 address and baud rate.
- Per-channel output range.
- Measurement period.
- Measurement window.
- Methane warm-up and polling periods.
- HTU21D humidity read period.
- DAC min, max, and fault current.
- RS485 termination state.

The shell exposes the same runtime record for field service:

```
argisense settings list
argisense settings get <name|alias>
argisense settings set <name|alias> <value>
argisense settings reset
```

Every shell write is validated by the settings module before it is saved to NVS. RS485 baudrate, data-bit, parity, stop-bit, and Modbus address writes are persistent immediately, but the running RS485 server uses the old transport parameters until the next reboot.

Recommended storage approach:

- Keep Zephyr Settings/NVS as the primary runtime settings backend.
- Keep AT24C512C available for future factory data, calibration export, or service logs that must live outside internal MCU flash.
- Add an EEPROM-backed settings mirror only if the product must store runtime settings outside MCU internal flash.
- Keep a settings version and CRC so invalid calibration can be detected.

External AT24C512C EEPROM

The schematic includes U4, `AT24C512C-SSHD-T`, connected to `MCU_I2C1_SCL` and `MCU_I2C1_SDA`. A0/A1/A2 are strapped for address `0x50`, and the part is powered from `+3V3_PRE`.

Firmware mapping:

Property	Value
DTS alias	eeprom0
Compatible	microchip,24c512, atmel,at24
I2C bus	i2c1
I2C address	0x50
EEPROM size	65536 bytes
Page size	128 bytes
Address width	16 bits
Write timeout	5 ms

Boot only checks device readiness and logs the mapping. It does not write to the EEPROM. Shell command `argisense eeprom [offset] [length]` temporarily powers the measurement rail and reads up to 64 bytes for bring-up diagnostics.

Current Implementation Status

Implemented:

- Out-of-tree Zephyr board targets for methane + pressure and future pH hardware profiles.
- MCUboot/sysbuild image layout with primary slot, secondary slot, and persistent settings storage.

- Power manager for sensor rails, DAC power, pressure protocol select, RS485 termination, and measurement-window sequencing.
- Zephyr Settings/NVS runtime configuration in `storage_partition`.
- Dynament Platinum methane Sensor API driver and AN0007 live-data-simple parser.
- MS5803-05BA pressure Sensor API driver with reset, PROM read, PROM CRC, D1/D2 ADC conversion, and compensated pressure/temperature output.
- HTU21D Sensor API driver with soft reset, no-hold humidity/temperature conversion, CRC validation, and cached application reads.
- AT24C512C external EEPROM mapping through Zephyr's standard AT24 EEPROM driver, with read-only ArgiSense shell diagnostics.
- GP8302 Zephyr DAC driver for two independent current-loop outputs.
- Modbus RTU register map v6 with live data, settings, MCUboot version fields, commands, diagnostics, and RS485 MCUboot image upload support.
- USB-C composite CDC ACM profile for console/shell plus MCUmgr firmware update through the `argisense-usb-update` snippet.
- Shell diagnostics for driver readiness, one-shot sensor reads, settings, and register inspection.
- Python/Tkinter RS485 service GUI under `tools/rs485_dfu` for auto-detect, firmware update, live sensor graphing, and runtime device configuration.
- Python/Tkinter USB-C MCUmgr GUI under `tools/usb_mcumgr`.
- Rollback-friendly MCUboot/sysbuild policy for bring-up and field service, with downgrade prevention currently disabled.

Remaining recommended work:

- Validate Dynament byte-stuff handling and timeout behavior on real hardware with long UART captures.
- Add HTU21D calibration or correction hooks if the final product requires humidity-compensated methane or pressure calculations.
- Decide whether humidity should become a user-visible primary value, a compensation-only value, or both.
- Add watchdog behavior and fault escalation after repeated sensor failures.
- Add hardware-in-loop tests for pressure, methane, humidity, RS485 writes, persistent settings, and DAC current-loop output.
- Decide the final production firmware rollback/downgrade policy before release.
- Validate current consumption in idle, measurement, RS485-active, and continuous-DAC-output modes.

Open Hardware Items

These details should be confirmed before final firmware lock:

- Exact Dynament methane ordered range: 0-5% volume, 0-100% volume, or 0-5-100% volume.
- Whether HTU21D is powered from `+3V3_PRE` or another rail in the final PCB.
- Exact pressure oversampling ratio and conversion timing policy.
- GP8302 register format, resolution, and alarm polarity.
- Whether `dac-alarm-gpios` is a shared alarm, only DAC0 alarm, or should become two separate alarm GPIOs.
- Whether either 4-20 mA output must be continuously driven while sleeping.
- Whether RS485 default baud should be 9600, 19200, or 115200.
- Whether RS485 termination should be controlled by firmware or fixed by installer configuration.